

Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Abstract

Pre-deployment dependability assessment of asynchronous distributed systems is hindered by the absence of runtime telemetry and by static code analysis’s blindness to communication topology. This paper presents Software-as-a-Graph (SaG), a static system analysis framework that constructs typed multigraphs from Architecture-as-Code manifests across Applications, Brokers, Topics, Execution Nodes, and Shared Libraries. SaG integrates two complementary pathways: a predictive pathway employing a relation-specific Heterogeneous Graph Transformer (HGT) with Quality-of-Service (QoS) edge encodings to forecast cascading failure blast radii, and an interpretable explanation layer grounded in ISO/IEC 25010 that attributes vulnerabilities to Availability (single points of failure) or Fault Tolerance (cascade hubs) to prescribe targeted architectural repairs.

Evaluated across seven synthetic scenarios and three open-source reference systems (Autoware.universe ROS 2, GCP Online Boutique, Train-Ticket): (1) Typed heterogeneous learning over the full multigraph transfers out of distribution ($\rho = 0.608$) where untyped learning over a projected view collapses ($\rho = 0.086$, $p = 0.008$); the two consume different substrates, so this margin conflates typing with multi-entity visibility. (2) Against a training-free QoS-weighted centrality baseline, the ranking margin is $+0.037$ and not statistically significant ($p = 0.64$); the critical-set margin ($F_1@K = 0.414$ vs. 0.380) is untested and within oracle seed noise. (3) SaG transfers zero-shot to the reference systems ($\rho = 0.688$ – 0.778). (4) An HGT forward pass costs 43.7 ms at $2,000$ components; including the topological features it consumes, the complete gate runs in 0.02 – 27.4 s — within pull-request CI/CD budgets. SaG bridges the Architecture–Code Gap, facilitating dependable and performant distributed software systems.

Keywords: Graph representation learning, Heterogeneous graph neural networks, Distributed systems dependability, Cascading failures, Static system analysis, Explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. Across diverse domains—from autonomous driving (ROS 2 [1]) and enterprise event streams (Apache Kafka [2]) to cyber-physical backbones (DDS [3]), IoT fleets (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM serving clusters—pub-sub decouples producers and consumers in space, time, and synchronization [7]. Components interact indirectly through intermediate message topics and brokers without maintaining direct static references. Furthermore, modern middleware specifications allow engineers to configure deployment-time Quality-of-Service (QoS) policies—such as reliability guarantees, durability, message priorities, and delivery deadlines—to govern how traffic behaves under peak load and network stress.

While this architectural decoupling confers elastic scalability and operational flexibility, it creates a formidable **visibility barrier** for system performance, reliability, and computational sustainability:

- **Indirect Failure and Degradation Pathways:** In traditional synchronous architectures (e.g., RESTful HTTP or gRPC), component interactions follow explicit caller–callee invocation paths. In asynchronous pub-sub and event meshes, publishers and subscribers share no direct references.

Cascading failures, queue head-of-line blocking, and backpressure propagate across hidden logical paths spanning brokers, shared topics, colocated execution nodes, and shared libraries.

- **Distinct Degradation Mechanisms:** Disturbances in complex distributed systems do not propagate in a uniform manner. They manifest either as *sequential cascades* (e.g., a slow subscriber causing broker queue saturation and upstream backpressure) or as *simultaneous blast radii* (e.g., a shared runtime library crash, memory exhaustion, or host machine outage instantly disabling multiple colocated services). Conventional architectural diagrams and static call graphs fail to represent these multi-layer dependencies.

Addressing these architectural vulnerabilities is most effective and cost-efficient **prior to deployment**, during design and Continuous Integration / Continuous Delivery (CI/CD), adhering to established foundational principles of dependable computing [8]. However, at design and build time, **no runtime telemetry, distributed tracing, or operational logs exist**. Consequently, software architects, performance engineers, and Site Reliability Engineers (SREs) face two fundamental questions without operational data:

1. *Which components, message topics, and communication links are systemically critical to system dependability and performance?*
2. *Why are they critical, and what specific architectural repair (such as replicating a message broker, decoupling an over-subscribed topic, or sandboxing a shared library) will most effectively eliminate that risk?*

Resolving these questions is equally vital for **system performance and computational sustainability**. When cascading failures reach production environments, they initiate vicious cycles of compute-intensive restart loops, retry storms with exponential backoff, cluster-wide failover thrashing, and severe tail-latency spikes. These pathologies squander massive CPU cycles, memory buffers, and network bandwidth on unproductive work, inflating cloud infrastructure costs, energy consumption, and carbon footprints. Proactive, design-time architectural hardening eliminates these systemic defects before software is deployed, preserving computational energy and protecting operational budgets.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

We formulate pre-deployment dependability and performance analysis around two distinct, complementary tasks:

1. **Failure-Impact Forecasting (Predictive Pathway) — the primary task.** We forecast dynamic cascading failure blast radii and identify critical components using a data-driven, relation-specific model over learned topological representations. Closed-form topological metrics capture broad connectivity cheaply; whether they also resolve multi-hop, relation-dependent cascade spread across heterogeneous channels is an empirical question, which we test directly against such a baseline (Section 7.1). Our predictive pathway is trained and evaluated against independent simulation ground truth as a ranking and critical-set identification model.
2. **Explainable Criticality Attribution (Explanation Layer) — what a rank alone cannot say.** A ranked shortlist indicates *where* risk lies, but not *how to fix it*. We therefore pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [9] and ISO/IEC 25019 [10]. This layer diagnoses the *qualitative root cause* of vulnerability—distinguishing, for instance, an unreplicated single point of failure from a high-coupling maintainability bottleneck—to guide concrete repairs. It serves strictly as an attribution model, not a ranking model.

This separation is architectural rather than merely presentational: both pathways operate on the same graph but share no parameters, and neither is trained on the other’s output. The coupling term that could connect them is disabled by default and reported only as an ablation (Section 4.2). Maintaining this independence allows SaG to identify components that are structurally central yet operationally low-impact—a nuanced diagnosis unattainable by either pathway alone.

Existing software engineering approaches fail to bridge what we define as the **Architecture–Code Gap**: *a distributed system can have pristine, 100% bug-free source code within each individual service, yet*

remain fragile to catastrophic global outages and tail-latency explosions due to hidden architectural single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts. Although classical architecture evaluation methods such as the Architecture Tradeoff Analysis Method (ATAM) [11, 12] identify architectural risks, and software studies analyze architectural technical debt [13] and bad smells [14], they rely on manual stakeholder elicitation rather than quantitative structural learning. Prevailing automated paradigms leave this gap unaddressed:

- **Static Code Analysis (SCA):** Tools such as SonarQube [15] inspect source code complexity [16], modularity, and object-oriented cohesion and coupling metrics (e.g., Lack of Cohesion in Methods [LCOM] and Coupling Between Objects [CBO]) [17, 18] within single services. However, SCA cannot observe the broader distributed network, message queues, or cross-host failure propagation.
- **Runtime Chaos Engineering:** Techniques like Chaos Monkey [19] and distributed tracing inject real faults into running staging or production environments. While effective for live systems, they require fully deployed infrastructure, carry operational risks, incur high computational and energy costs through repeated test executions, and arrive too late to guide initial architectural design.
- **Homogeneous Graph Centrality Metrics:** Standard network metrics (betweenness, PageRank, degree) [20, 21, 22, 23] flatten systems into simple, unweighted graphs. They treat all connections identically, failing to distinguish between an asynchronous message topic, a shared library, and an execution host.

Furthermore, while machine learning has demonstrated remarkable success across software engineering, contemporary AI approaches applied to system dependability often function as uninterpretable black boxes. Deep neural models frequently output scalar risk scores or latent embeddings without providing transparent, actionable rationales for their predictions. In mission-critical software engineering, an opaque risk score is inadequate: developers and SREs cannot refactor code or reconfigure infrastructure without understanding *why* a component is vulnerable and *which* architectural mechanism is compromised.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while overcoming the black-box AI challenge, this work introduces **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention lets a `USES` edge into a shared library propagate differently from a `PUBLISHES_TO` edge into a topic. It forecasts cascading blast radii, ranks critical components, and outputs per-relationship criticality alongside multi-task quality outputs (Section 4), at 43.7 ms per 2,000-component system once the topological node features it consumes have been computed (Section 7.5).
4. **Explainable Quality Attribution (Explanation Layer):** To explain *why* a flagged component is critical, SaG combines code-level SCA metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model (Section 5). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), pointing to distinct repairs. Because it is a linear, propagation-free aggregate by design, it explains *why* a component is vulnerable rather than how far a cascade travels. Its standalone rank correlation is correspondingly modest ($\rho = 0.195$, Section 7.1)—a direct reflection of its explanatory scope rather than a tuning defect.

To ensure methodological rigor, SaG enforces a strict **input–label independence guarantee**: learned models and attribution baselines operate exclusively on the analytical graph G_{analysis} , while ground-truth failure impacts are generated by independent discrete-event simulators operating on the raw structural topology $G_{\text{structural}}$ (Section 4.4).

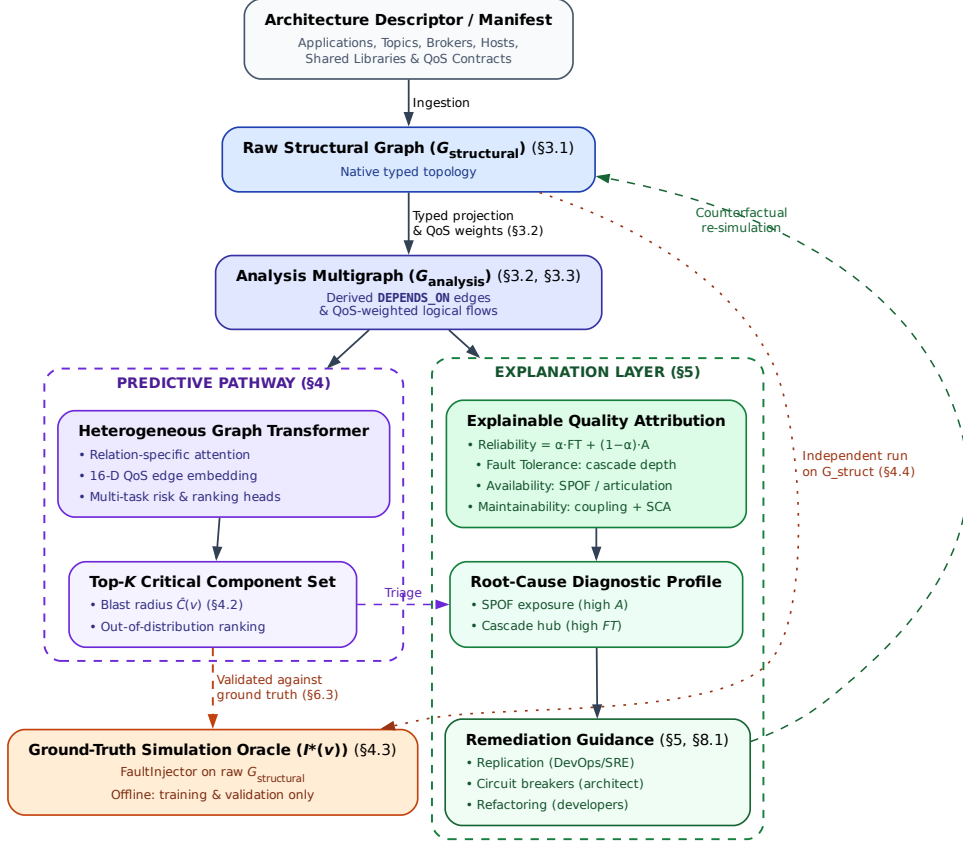


Figure 1: End-to-end architecture of the Software-as-a-Graph (SaG) framework. A shared front end (manifest ingestion → typed multigraph → QoS-weighted **DEPENDS_ON** projection → typed node features) feeds two deliberately separate pathways. The **predictive pathway** (Section 4) is the primary one: it produces a ranked critical set and per-relationship criticality, and is the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not, and it is strictly an offline training-and-validation component, never a dependency of online inference. The **explanation layer** (Section 5) then produces a standards-grounded quality profile for what the predictor flagged, and the remediation it implies; it explains *why* a component is fragile and is not a ranking model. The single link between them is triage rather than data flow: the architect applies the explanation to whatever the predictor flagged. The two share no parameters, and the oracle runs on $G_{\text{structural}}$ alone, never on the graph the predictors see (Section 4.4). The remediation guidance closes a loop of its own: each candidate edit is re-simulated on its own mutated copy of $G_{\text{structural}}$ and kept only if it beats the simulator’s own seed-to-seed noise, before being accepted.

Rationale for Graph Learning vs. Direct Simulation. Given that discrete-event simulation $I^*(v)$ defines ground-truth criticality in this study, it is reasonable to ask: *why train a graph neural network rather than relying solely on simulation sweeps or closed-form heuristics?* While a simulator could identify critical components in an existing system, four practical reasons make our hybrid graph learning and attribution framework essential:

1. **Handling Unmeasured Infrastructure Components:** Discrete-event simulators only inject faults into active application processes, leaving passive infrastructure (such as message topics or host nodes) without direct simulation labels (30% to 47% of components per system). The learned GNN generalizes across both labeled and unmeasured entities through message passing.
2. **Computational Efficiency and CI/CD-Viable Speed:** Cascade simulations are computationally intensive, stochastic, and sensitive to seeds and propagation thresholds (label standard deviation across seeds reaches 0.416). The neural network learns a smooth, threshold-marginalized representation and re-scores an already-analysed architecture in 43.7 ms on a 2,000-component system. The forward pass is not the whole cost: the topological features it consumes are produced by a preceding deterministic analysis stage, placing the complete gate at 0.02–27.4 s across our corpus. That budget — not the forward pass alone — is what makes per-commit architectural gating practical within CI/CD pipelines (Section 7.5).
3. **Diagnostic Explainability (Resolving the Black Box):** Simulators and unaugmented GNNs both return only scalar impact scores or ranks; neither reveals root causes in standardized software engineering terms. While a simulator can show that a subscriber lost a feed, it cannot determine whether the component is fragile due to an unreplicated single point of failure or an error-cascade hub—diagnoses that require fundamentally different remediations (host/broker replication versus topic decoupling and circuit breakers). Our ISO-grounded RM model supplies that missing layer: once the predictor identifies a critical component, the diagnostic path explains which quality characteristic is compromised and which architectural repair applies.
4. **Pre-Deployment Zero-Telemetry Transfer:** Dynamic simulators require runnable containers, operational mocks, or configured communication harnesses to execute message exchanges. Heterogeneous graph learning enables zero-shot inductive evaluation directly from Architecture-as-Code manifests during continuous integration, assessing topological fragility and performance bottlenecks before provisioning any runtime infrastructure.

1.4. Research Questions

This empirical study investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) yield better failure predictions than homogeneous graph models, and does that advantage hold on architectures the model has never seen?*

RQ3 (QoS Encoding, Calibration and Sensitivity): *How do middleware Quality-of-Service policies, the declared weighting constants of the explanation layer, the choice of simulation oracle, and propagation-threshold and normalization settings affect forecasting performance, stability, and explainability?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to authentic, real-world distributed systems across autonomous driving (ROS 2) and cloud-native microservice architectures?*

RQ5 (Analysis Cost and Computational Sustainability): *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that computational footprint, and does the resulting budget enable sustainable, per-commit architectural gating?*

1.5. Key Contributions

This paper presents four principal contributions:

1. **Heterogeneous Graph Learning for Pre-Deployment Dependability:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code alone, with a 16-dimensional edge feature vector carrying 7 QoS dimensions and multi-task heads for component and relationship criticality (Section 4). Our central empirical claim concerns cross-architecture transfer: typed heterogeneous learning over the full multigraph generalizes out of distribution to architectures it never trained on ($\rho = 0.608$), where untyped learning over a projected view collapses ($\rho = 0.086$, $p = 0.008$; Section 7.2). Because the typed and untyped variants consume different substrates, this margin reflects relation typing and multi-entity topological visibility jointly; an untyped control on the identical multigraph is required to separate them, and we state this as the principal open control on our central claim (Sections 6.2.1 and 7.2). Simultaneously, we establish the honest empirical boundary: against an unparameterized QoS-weighted centrality baseline (Topo-QoS), out-of-distribution ranking is matched rather than surpassed ($\Delta\rho = +0.037$, $p = 0.64$), and the critical-set margin ($F_1@K = 0.414$ vs. 0.380) carries no significance test and falls within the simulation oracle’s own seed-to-seed reproducibility (Section 8.3).
2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures, supplying the typed substrate the predictor consumes (Section 3).
3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability model grounded in ISO/IEC 25010/25019 that turns the predictor’s ranked output into an actionable diagnosis, separating single-point-of-failure exposure from error-propagation depth—two distinct failure modes requiring different repairs (Section 5).
4. **Empirical Benchmark, Real-World Validation, and Sustainability Characterization:** A rigorous evaluation across seven synthetic topologies (1,545 components) and three real-world systems (Autoware.universe ROS 2, GCP Cloud Microservices, Train-Ticket; 225 components) under strict input–label independence, establishing both where typed graph learning delivers decisive advantages and the boundary where it does not, with a per-stage cost profile locating the pipeline’s computational cost in its deterministic graph-analysis stage rather than in the learned model (Sections 6–7).

Relationship to the authors’ prior work. An earlier, shorter version of this work was presented at a peer-reviewed conference [24], focusing on the preliminary typed multigraph formulation and the deterministic quality-attribution model using only the synthetic corpus. This manuscript is a substantially extended version containing over 70% new technical contributions, fully meeting the Journal of Systems and Software extension policy. Major new contributions include: (1) the complete predictive pathway: the Heterogeneous Graph Transformer, its 16-D continuous-categorical QoS edge encoding, the multi-task masked-loss heads, and all learned results in Section 7; (2) the inductive leave-one-scenario-out (LOSO) protocol and cross-architecture transfer analysis supporting the central typing claim (Section 7.2); (3) zero-shot empirical evaluations across three open-source reference systems (Autoware.universe ROS 2, GCP Online Boutique, and Train-Ticket; Section 7.4); (4) an extensive computational cost and sustainability characterization answering RQ5 (Section 7.5); (5) a formal four-oracle convergent-validity taxonomy and strict input–label independence guarantee preventing data leakage (Sections 4.3–4.4); and (6) global sensitivity analysis across all ten weight constants using Morris screening and Dirichlet simplex sampling (Section 7.3). Material retained from the conference version is limited to preliminary formalisms in Sections 3 and 5, both of which have been thoroughly restructured and expanded. No companion manuscript from this work is under consideration elsewhere.

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work on distributed systems dependability, performance engineering, static system analysis, and graph representation learning. Section 3 formalizes the Software-as-a-Graph architectural model, the dependency projection rules, and the typed node features consumed by both pathways. Section 4 presents the Heterogeneous Graph Transformer, its multi-task heads, the simulation oracles that supply its labels, and the input–label independence guarantee. Section 5

introduces the interpretable RM explanation layer. Section 6 describes the experimental setup, benchmark corpus, and evaluation protocols. Section 7 presents empirical results for RQ1–RQ5. Section 8 discusses architectural implications, performance and computational sustainability, threats to validity, limitations, and concluding remarks.

2. Related Work

This work builds upon and connects four foundational research areas: (1) dependability, performance, and sustainability in distributed software systems; (2) static code and system analysis; (3) software quality measurement and multi-criteria evaluation; and (4) graph representation learning and explainable AI (XAI).

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization, enabling elastic scalability and high throughput [7]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies that regulate message durability, transport reliability, priorities, and delivery deadlines. In cloud-native microservice meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, directly shaping tail latencies, throughput bottlenecks, and hardware resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive backpressure throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [19] inject simulated faults or artificial latency into staging or production clusters to observe degradation and recovery behaviors. While runtime fault injection delivers valuable operational validation, it exhibits critical limitations:

- **Requires live infrastructure:** It demands fully provisioned, operational clusters, rendering it unusable during pre-deployment architectural design and early CI/CD phases.
- **Operational risk:** Injecting faults on staging or production systems carries operational risks, including accidental service disruptions and customer impact.
- **High computational and carbon cost:** Running multi-node chaos sweeps, canary deployments, and distributed tracing consumes extensive CPU, memory, and networking resources across cloud clusters, conflicting with modern sustainable computing mandates.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities and performance bottlenecks directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned. This approach enables zero-runtime, environmentally sustainable architectural gating within CI/CD pipelines.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [15]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [16], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [17, 18], and code duplication to flag internal code smells and defect-prone modules [25, 26, 27, 28]. However, SCA cannot observe runtime communication topology: it is blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

To bridge this “Architecture–Code Gap,” **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This allows engineering teams to detect structural anti-patterns [29, 30] and architectural technical debt [31] early during continuous integration (CI/CD) [32, 33], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [9] and the **ISO/IEC 25019:2023** Quality-in-Use model [10]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Fault Tolerance and Availability.
- **Maintainability:** The degree of effectiveness and efficiency with which software can be modified, comprising Modularity, Modifiability, and Analysability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behavior (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [34, 35]. In distributed architectures, architectural debt (such as over-centralized message topics or unreplicated brokers) degrades internal quality and precipitates severe external performance bottlenecks, queue congestion, and outages.

Aggregating multi-attribute structural metrics into an auditable quality score constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [36] delivers a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) to ensure mathematical soundness in weighting models. This study applies AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, in conjunction with learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, such as degree, closeness, betweenness centrality [20, 22], articulation points, and PageRank [21, 23]. Foundational studies on network robustness [37], cascading overloads [38], and interdependent networks [39] model how disruptions propagate across connected systems. However, standard network metrics suffer from two major limitations when applied to software architectures:

1. **Dimensional Collapse:** A single centrality scalar cannot distinguish *why* a component is critical—for instance, whether it is a single point of failure (SPOF), an error-propagating cascade hub, or an over-shared library.
2. **Semantic Collapse:** Standard metrics treat all nodes and edges identically. They conflate fundamentally different architectural entities, such as an asynchronous message topic, a shared library, and a physical execution host.

To overcome the limits of hand-engineered metrics, recent research has applied machine learning to network vulnerability (e.g., FINDER [40], DrBC [41], and PowerGraph [42]). However, most available models rely on **homogeneous message passing** (GCN [43], GraphSAGE [44], GAT [45]), which averages signals across all connections indiscriminately. Because distributed software architectures are inherently **heterogeneous** (comprising distinct entity types and relationship rules), homogeneous models blur critical architectural boundaries and fail to generalize out-of-distribution.

Heterogeneous Graph Neural Networks (RGCN [46], HAN [47], HGT [48], MAGNN [49]) resolve this by employing relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** architecture [48] to preserve typed relational semantics when forecasting cascading failure blast radii and performance degradation.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle in applying modern AI to software engineering is the **black-box barrier**: deep neural models output risk scores or continuous embeddings without explaining underlying structural causality. In production software engineering, uninterpretable risk rankings hinder actionable decision-making: developers and SREs cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [50] and PGExplainer [51], identify influential subgraphs through edge masking or parameterized learning. Although useful, these methods explain the model using internal latent representations rather than standardized software engineering concepts. SaG resolves this limitation through a decoupled dual-pathway design: the predictive HGT pathway reveals typed mutual-attention distributions indicating *which* architectural relations propagated the cascade (Section 7.3), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), translating raw predictions into actionable, cost-effective remediations.

Table 1: Comparison of dependability and performance analysis paradigms for distributed systems.

Paradigm	Lifecycle Stage	Topology-Aware	Multi-Typed	Explainable (XAI)	Zero-Runtime Needed	CI/CD Energy Cost
Static Code Analysis (SCA) [16, 17]	Pre-Deployment	No (Single Service)	No	Yes (Code Smells)	Yes	Minimal (Seconds)
Chaos Engineering [19]	Post-Deployment	Yes (Live Cluster)	Partial	Partial (Logs/Traces)	No (Requires Cluster)	High (Cluster-Hours)
Network Centralities [20, 21]	Pre-Deployment	Yes (Flat Graph)	No	No (Single Scalar)	Yes	Low (Seconds)
Homogeneous GNNs [43, 45]	Pre-Deployment	Yes (Flat Graph)	No	No (Black-Box)	Yes	Low (Milliseconds)
Software-as-a-Graph (SaG)	Pre-Deployment	Yes (Multigraph)	Yes (5 Types)	Yes (ISO/IEC RM)	Yes (Manifest-Based)	Minimal (44 ms Forward)

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is formally modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \quad (2)$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \rightarrow [0, 1]$ and $w_E : E \rightarrow [0, 1]$ are weighting functions representing entity criticality and connection strength.

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), linking code-level fragility directly to topological analysis.

Table 2: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	<code>/sensor/lidar</code> , <code>orders.payment.completed</code>
Node (V_{node})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	<code>librdkafka</code> , OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	Component publishes messages to topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component consumes messages from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Node	Process is hosted on physical/virtual host
CONNECTS_TO	Node $\rightarrow$ Node	Physical network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library dependency

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For instance, a **RELIABLE** topic with **TRANSIENT_LOCAL** durability binds communicating services substantially more tightly than a **BEST_EFFORT** telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75, 0.15, 0.10) \quad (3)$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (4)$$

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data persists across restarts and network partitions. Reliability and transport priority both govern in-flight delivery quality, with reliability receiving higher weight because unconditional delivery guarantees precede message scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix with a small, non-zero consistency ratio ($CR \approx 0.016 \leq 0.10$). The modulators are logarithmically compressed: $\text{SizeNorm}(t) = \log_2(1 + \text{bytes})/20$ (a 1 MiB design envelope, representing the practical DDS sample ceiling before RTPS fragmentation dominates) and $\text{FreqNorm}(t) = \log_{10}(1 + \text{Hz})/3$. The weight $w(t)$ is clamped to $[0.01, 1]$ ensuring that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (**PUBLISHES_TO**, **SUBSCRIBES_TO**, **ROUTES**) inherits $w_E(e) = w(t)$ along with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination whose sensitivity is evaluated directly in Section 7.3.2. A prior KiB-based SizeNorm divisor of 50 implied an unintended ~ 1 EiB envelope and left the term realizing only $\sim 2\%$ of α 's declared 15% budget on the evaluation corpus's actual payload sizes (32 B–32 KiB); the byte-based 1 MiB envelope corrects this, ensuring that α 's contribution remains meaningful.

3.2.1. Logical Dependency Projection (**DEPENDS_ON**)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, **DEPENDS_ON**, directed from *dependent* to *dependency* (“if target fails, source is impacted”):

Rules 1 and 2 aggregate the set of topics T connecting a component pair using a probabilistic union rather than a maximum [52, 53, 54]. This guarantees that additional parallel failure vectors increase coupling monotonically while keeping $w \in (0, 1]$. Rule 5 applies the harmonic mean $H(x, y) = 2xy/(x + y)$ [55] to combine the consuming Application's and the shared Library's vertex weights, balancing caller and

Table 3: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	<code>app_to_app</code>	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive <code>USES</code>)	$1 - \prod_{t \in T} (1 - w(t))$
2	<code>app_to_broker</code>	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	<code>node_to_node</code>	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	<code>node_to_broker</code>	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	<code>app_to_lib</code>	Application $\rightarrow$ Shared Library it <code>USES</code>	$H(w_V(\text{app}), w_V(\text{lib}))$
6	<code>broker_to_broker</code>	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{node})$

dependency criticality. Rules 3 and 4 assign the maximum weight among component-level dependencies crossing the host boundary.

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A foundational principle of the SaG model is distinguishing between two fundamentally different degradation modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers suffer message starvation. The failure propagates hop by hop through message queues and topic buffers.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single shared-fate event.

Preserving architectural entity types and relation-specific projection rules enables SaG to model both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is intentionally the only symmetric projection rule. It does not imply that one broker functionally depends on another, but rather that two brokers colocated on the same host share that host’s physical failure domain. This follows the same simultaneous-blast principle as Rule 5, which is why the derived weight equals the shared Node’s weight and the relation is bidirectional. In production middleware deployments, colocated brokers compete for host resources (CPU cores, page cache, file descriptors, and NIC bandwidth); a host outage takes down all colocated instances simultaneously. Operational best practices for Kafka, RabbitMQ, and EMQX recommend distributing brokers across fault domains. Rule 6 does not model directional intra-cluster broker coupling (e.g., partition replication, controller quorum election, federation, or shovel links), which do not require physical colocation; extending the schema to capture these interactions is reserved for future work. In our evaluation corpus, Rule 6 applies in four of eight cached scenarios and contributes only 12 directed edges among 1,770 components. Because the simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as `PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, and `USES`). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis} .

G_{analysis} is further structured into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling evaluation of criticality at subsystem levels, consistent with hierarchical frameworks such as MIL-STD-498 [56].

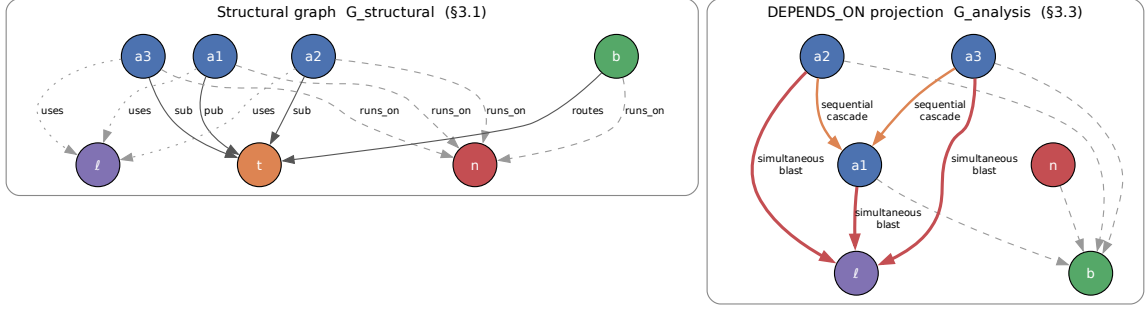


Figure 2: Running example: the raw structural graph (left) and the `DEPENDS_ON` projection derived from it (right). The projection makes implicit runtime dependencies explicit—a subscriber depends on the publishers of its topics even though no structural edge joins them—while the simulators continue to operate on the structural view alone.

3.4. Typed Node Feature Encoding

Both pathways read the same typed node properties from G_{analysis} : the predictive pathway (Section 4) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5) aggregates them into its quality profile. SaG extracts feature vectors tailored to the five entity types:

- **Application ($|V_{\text{app}}|$, 23 dims):** Indices 0–17 represent shared topological metrics (in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score, bridge load). Indices 18–22 capture source code metrics extracted via Static Code Analysis (SCA): Lines of Code (LOC), Cyclomatic Complexity, Martin’s Instability metric ($I_{\text{code}} = \frac{C_e}{C_a + C_e}$, where C_e is efferent coupling and C_a is afferent coupling) [57], Lack of Cohesion in Methods (LCOM), and composite Code Quality Penalty (CQP).
- **Library ($|V_{\text{lib}}|$, 25 dims):** Shared topological (0–17) and code quality (18–22) metrics as Application, plus two library-specific structural drivers (indices 23–24): the normalized size of the transitive reverse-USES closure and the normalized count of distinct subscribers reachable from published topics within that closure—the two structural drivers of a library’s blast radius under cascade rules that code-quality metrics alone cannot capture.
- **Broker ($|V_{\text{broker}}|$, 19 dims):** Indices 0–17 shared topological metrics; index 18 represents normalized queue buffer capacity.
- **Topic ($|V_{\text{topic}}|$, 22 dims):** Indices 0–17 shared topological metrics; indices 18–21 capture publisher count, subscriber count, log message frequency $\log(1 + \text{freq})$, and ordinal QoS criticality.
- **Infrastructure Node ($|V_{\text{node}}|$, 20 dims):** Indices 0–17 shared topological metrics; indices 18–19 capture normalized CPU core allocation and physical memory (RAM).

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems is inherently non-linear, multi-hop, and relation-dependent. Outages propagate not merely based on neighbor count, but through architectural relations and dependencies extending multiple hops beyond the initial fault. Whether a closed-form combination of standard centrality metrics can capture these compound dynamics is an empirical question rather than a settled one. The primary predictive pathway of Section 1.2 therefore employs a learned graph model, and

Section 7.1 evaluates it against exactly such a closed-form baseline — which, on out-of-distribution ranking, it does not significantly surpass.

This section details the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input-label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON), we employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [48], implemented within PyTorch Geometric [58], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture ensures that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous Quality-of-Service (QoS) constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$:

- **Index 0 (Scalar Coupling Weight):** $w_E(e) \in (0, 1]$, defined in Section 3.2 (inherited from $w(t)$ for structural pub/sub edges; or derived via probabilistic union, lifted maximum, or harmonic mean for projected DEPENDS_ON edges).
- **Index 1 (Path Count):** Normalized count of simple paths traversing edge e in G_{analysis} .
- **Indices 2–8 (Relation Type One-Hot):** 7-bit one-hot encoding for the structural and derived relations (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON).
- **Indices 9–15 (Explicit QoS Dimensions):** 7 continuous-categorical QoS profile attributes, active for PUBLISHES_TO and SUBSCRIBES_TO edges (zeroed for other edge types):
 9. *Reliability score* (0.0 = best-effort, 1.0 = reliable).
 10. *Durability score* (0.0 = volatile, 0.5 = transient local, 0.6 = transient, 1.0 = persistent).
 11. *Message priority* (0.0, 0.33, 0.66, 1.0 for low, medium, high, urgent).
 12. *Deadline active flag* (0/1).
 13. *Log deadline* $\log_{10}(1 + \text{deadline_ns}/10^6)$.
 14. *Log max blocking time* $\log_{10}(1 + \text{max_blocking_ms})$.
 15. *QoS heterogeneity flag* (1 if the edge’s QoS triple deviates from the scenario’s modal profile, 0 otherwise).

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}}e_{uv}$. Prior to relational attention computation, this projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D -dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v)) \quad (5)$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention across H heads:

$$\text{Attn}(u, e, v) = \underset{\forall u \in \mathcal{N}(v)}{\text{Softmax}} \left(\frac{K(u)W_{\text{att}, \phi(e)}Q(\tilde{h}_v)^\top}{\sqrt{D/H}} \right) \quad (6)$$

$$\text{Msg}(u, e, v) = V(u)W_{\text{msg}, \phi(e)} \quad (7)$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream back-pressure simultaneously, message passing is executed over both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$).

4. **Residual Aggregation and Layer Normalization:** Target node representations are updated across layers $l \in \{1, \dots, L\}$ via residual connections and layer normalization:

$$h_v^{(l)} = \text{LayerNorm} \left(h_v^{(l-1)} + \text{Dropout} \left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \quad (8)$$

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end using AdamW with initial learning rate $\eta = 3 \times 10^{-4}$, weight decay 10^{-4} , and dropout probability 0.10. Learning rates follow a cosine annealing schedule with warm restarts (CosineAnnealingWarmRestarts, $T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\text{min}} = 3 \times 10^{-6}$). Training executes for a maximum of 300 epochs with early stopping governed by a patience of 30 epochs monitored on validation loss over labeled nodes. Inductive subgraphs are isolated strictly via training masks to prevent information leakage across validation/test splits.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The combined optimization objective integrates regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \quad (9)$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is the ListMLE ranking loss [59], $\mathcal{L}_{\text{pairwise}}$ is margin-ranking loss, and $\mathcal{L}_{\text{consistency}} = \text{MSE}([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}})$ regresses predicted heads toward the diagnostic pathway’s baseline (Section 5) on unlabeled nodes. Headline results use $\lambda_{\text{RM}} = 0$, guaranteeing that the predictive and explanatory pathways remain strictly independent.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via **FaultInjector**) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is never used as a training label to avoid circular supervision. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \quad (10)$$

This mask ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

4.2.2. Domain-Reweighted Criticality (Q_{domain})

To ground predictions in ISO/IEC 25019 Context of Use ($\vec{\omega} = [q_R, q_M]^\top$), the composite score can be evaluated as:

$$Q_{\text{domain}}(v) = q_R \cdot \hat{R}(v) + q_M \cdot M_{\text{static}}(v) \quad (11)$$

where $M_{\text{static}}(v)$ is obtained directly from the structural analyzer’s maintainability baseline, combining learned dynamic reliability with static source-code maintainability. Because maintainability is unobserved in dynamic simulation ($m = [1, 0]$), headline results report $\hat{I}^*(v)$ directly, while domain reweighting sensitivity is evaluated against the static RM baseline in Section 7.3.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle ($I^*(v)$):** Implemented via **FaultInjector**, this oracle simulates node crashes at component $v \in V$, propagates cascading outages across dependent topics, brokers, and network links via breadth-first dynamic traversal, and calculates the fraction of surviving subscriber feeds severed. Publisher loss is weighted by message publication rate, and resulting feed losses are scaled by a QoS ladder ($\times 1.2$ for **RELIABLE**, $\times 1.15$ for high/urgent priority, $\times 1.05$ for medium) before clamping to $[0, 1]$. $I^*(v) \in [0, 1]$ serves as the **primary continuous target label** for training and evaluating GNN predictors.
- **Multi-Metric Composite Oracle ($I_{\text{comp}}(v)$):** Implemented via **FailureSimulator**, this oracle evaluates a multi-faceted failure impact vector:

$$I_{\text{comp}}(v) = 0.35 \cdot \Delta\text{Reachability} + 0.25 \cdot \Delta\text{Fragmentation} + 0.25 \cdot \Delta\text{Throughput} + 0.15 \cdot \Delta\text{FlowDisruption} \quad (12)$$

where each term is weighted by operational severity $s(t) = w(t) \cdot \text{rate}(t)$. $I_{\text{comp}}(v)$ serves as the canonical oracle for architectural quality gate verification.

- **Dynamic Queue-Flow Oracle ($I_{\text{dyn}}(v)$):** Implemented via **MessageFlowSimulator** using the SimPy framework [60], this oracle simulates message emission rates, stochastic network latencies, broker buffer saturation, and queue drops under fault injection. It extracts the drop in delivered message rate suffered by surviving consumers.
- **Change-Propagation Oracle ($I_M(v)$):** Implemented via **ChangePropagationSimulator**, this oracle executes a deterministic breadth-first traversal of the reversed dependency graph to quantify maintenance change impact:

$$I_M(v) = 0.45 \cdot \text{ChangeReach}(v) + 0.35 \cdot \text{WeightedChangeImpact}(v) + 0.20 \cdot \text{NormalizedChangeDepth}(v) \quad (13)$$

- **Relationship (Edge) Removal Oracle ($I_{\text{edge}}(u, v)$):** Evaluates the systemic impact of severing an individual dependency or communication channel while keeping endpoint components operational. Writing $\bar{I}_{\text{comp}}(G) = |V|^{-1} \sum_{v \in V} I_{\text{comp}}(v; G)$ for the mean composite impact over a graph G :

$$I_{\text{edge}}(u, v) = \bar{I}_{\text{comp}}(G \setminus \{(u, v)\}) - \bar{I}_{\text{comp}}(G) \quad (14)$$

Withholding Declared Topic Criticality from Labels. **FailureSimulator** supports blending a declared **Topic.criticality** label into its severity term, but this is explicitly disabled. Because **Topic.criticality** is an input feature to the GNN (**topic_qos_criticality_ord**), allowing an oracle to consume it would score the predictor against a transformation of its own input features.

Primary Oracle Declaration and Role Assignment. Because the three reliability-facing oracles (I^* , I_{comp} , I_{dyn}) measure distinct operational constructs, we designate $I^*(v)$ (**FaultInjector**) as the primary

oracle for all predictive ranking results (Tables 7–9, RQ1–RQ3). $I_{\text{comp}}(v)$ is reserved for Validate-stage quality gates, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe, and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity and Critical-Set Bounds. Measured across seven benchmark scenarios and five random seeds on the Application population, the mean Spearman rank correlation is $\rho = 0.883$ for (I_{dyn}, I^*) , $\rho = 0.468$ for (I_{comp}, I^*) , and $\rho = 0.465$ for $(I_{\text{comp}}, I_{\text{dyn}})$. The strong rank agreement ($\rho = 0.883$) between the behavioral queue-flow oracle and the topological cascade injector provides independent convergent evidence across distinct simulation paradigms. However, agreement on the top- K critical set ($K = 0.2n$) is more conservative (mean Jaccard overlap of 0.42 for the strongest pair vs. 0.111 expected by chance), highlighting the intrinsic sensitivity of discrete thresholding in non-linear cascades. Consequently, results established against one oracle are never transferred to another; every evaluation metric explicitly references its underlying simulation oracle.

4.4. Input-Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (**FaultInjector**, **FailureSimulator**, **MessageFlowSimulator**).

No simulation outputs, failure trace histories, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 answers *where* to act. It does not answer *what to do*, and neither does the oracle that scores it — both return impact, not cause attributed in standardized quality terms. A component may be critical because it is a single point of failure, because it propagates errors widely, or because it is a high-coupling maintenance bottleneck. These diagnoses call for distinct architectural repairs: replicating the host or broker, decoupling the topic, or refactoring the module. This section presents the layer supplying that diagnosis. SaG decomposes component and relationship criticality into a standards-grounded quality profile, computed over the same typed node features (Section 3.4) but sharing no parameters with the predictor, and applied to whatever the predictor flagged — triage rather than data flow (Figure 1).

5.1. Grounding in ISO/IEC Standards

In accordance with **ISO/IEC 25010:2023** (Product Quality Model) [9], **ISO/IEC 25019:2023** (Quality-in-Use) [10], and **ISO/IEC 25022:2016** (Measurement of Quality-in-Use) [61], SaG formalizes two primary criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system’s capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated across two orthogonal quality characteristics: **Reliability** (R) and **Maintainability** (M). In distributed systems, degradation of Reliability and Maintainability directly precipitates runtime performance collapse and computational energy waste. Specifically, components with high Fault Tolerance risk (FT , cascade hubs) cause severe message queue accumulation, head-of-line blocking, and tail-latency amplification during partial outages. Conversely, single points of failure (high Availability risk, A) trigger failover storms, connection thrashing, and retry loops with exponential backoff that dissipate substantial CPU and network bandwidth without completing useful work.

Table 4: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	Fault Tolerance (FT)	How broadly does failure propagate?	Reverse PageRank on G^T , in-degree, cascade depth	Reliability Eng.: add redundancy, circuit breakers
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, connectivity degradation	DevOps/SRE: replicate host/broker
Maintainability (M)	Modularity/Modifiability	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling-risk imbalance, inverse clustering	Architect: refactor code, decouple

Coverage Scope: SaG focuses specifically on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, such as ISO 26262 Automotive Safety Integrity Level [ASIL] ratings) and Security (which requires explicit threat models, such as STRIDE [Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege]) fall outside purely structural topology analysis and are reserved for domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [36]:

1. **Fault Tolerance** ($FT(v)$): Measures error cascade potential on the transpose graph G_{analysis}^T (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \quad (15)$$

where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v)$ is the enhanced Cascade Depth Potential term.

2. **Availability** ($A(v)$): Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component’s own QoS weight:

$$A(v) = 0.2563 \cdot \text{AP}_c^{\text{dir}}(v) + 0.1998 \cdot \text{QSPOF}(v) + 0.1998 \cdot \text{BR}(v) + 0.2563 \cdot \text{CDI}(v) + 0.0878 \cdot w(v) \quad (16)$$

where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted Single Point of Failure severity, $\text{BR}(v)$ is Bridge Ratio (edge-level irrecoverability), $\text{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is the component’s intrinsic QoS weight.

3. **Reliability** ($R(v)$): Blends Fault Tolerance and Availability hierarchically:

$$R(v) = \alpha \cdot FT(v) + (1 - \alpha) \cdot A(v), \quad \alpha = 0.36 \quad (17)$$

Intra-dimension pairwise comparison matrices are audited against Saaty’s consistency ratio and measure $CR = 0.001$ (Fault Tolerance), $CR = 0.001$ (Availability), and $CR = 0.000$ (Maintainability) — all well within the $CR \leq 0.10$ acceptability threshold. The shipped intra-dimension weights apply a $\lambda = 0.70$ shrinkage blend between the raw AHP-derived vector and a uniform prior (Section 7.3 reports ranking sensitivity to λ).

4. **Maintainability** ($M(v)$): Evaluates structural coupling combined with code-level static analysis across five terms — betweenness, QoS-weighted efferent coupling, the Code Quality Penalty, an afferent/efferent coupling-risk imbalance term, and inverse clustering:

$$M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v)) \quad (18)$$

where $\text{BT}(v)$ is Betweenness Centrality, $w_{\text{out}}(v)$ is QoS-weighted efferent coupling (out-degree), $\text{CQP}(v)$ is the Code Quality Penalty, $\text{CouplingRisk}_{\text{enh}}(v)$ is an afferent/efferent coupling-risk imbalance term, and $\text{CC}(v)$ is the local Clustering Coefficient.

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \quad (19)$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \quad (20)$$

Components are categorized into adaptive criticality tiers using box-plot quartile thresholds:

- **CRITICAL:** $Q > Q_3 + 1.5 \cdot \text{IQR}$
- **HIGH:** $Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$
- **MEDIUM:** $Q_1 < Q \leq Q_3$
- **MINIMAL:** $Q \leq Q_1$

This provides actionable diagnostics: a service scoring high on A but low on FT is diagnosed as a pure SPOF requiring horizontal replication, whereas a service scoring high on FT is an error cascade hub requiring circuit breakers, queue rate limiting, and bulkhead isolation. Remedying these targeted vulnerabilities not only restores architectural dependability but directly improves performance efficiency and curtails energy-intensive restart storms.

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 1,770 components distributed across ten distinct system architectures, as detailed in Table 5:

Table 5: Experimental evaluation corpus.

Dataset / Architecture	System Paradigm	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Autonomous Vehicle (AV)	ROS 2 Cyber-Physical	152	80	40	4	8	20	797
Enterprise Pub-Sub	Kafka Event Mesh	520	300	120	10	40	50	3,245
Financial Trading	Low-Latency Pub-Sub	124	60	35	5	6	18	580
Healthcare Integration	HL7/FHIR Event Mesh	98	50	25	3	8	12	400
Hub-and-Spoke Enterprise	Broker-Centric Messaging	139	70	30	2	12	25	797
IoT Smart City	MQTT Telemetry Mesh	326	200	80	6	30	10	1,322
Microservices Mesh	Cloud-Native Services	186	90	45	6	15	30	680
Autoware.universe [62]	Real-World ROS 2 Autoware	75	32	24	3	6	10	179
Cloud Microservices [63]	Real-World GCP Boutique	60	22	20	4	6	8	128
Train-Ticket [64]	Real-World Microservices	90	41	30	3	8	8	162
Total		1,770						

Here, $|V|$ is the sum of all five entity-type counts per scenario, totaling exactly 1,770 components. $|E|$ counts every raw structural relationship instance recorded in the scenario specification (**PUBLISHES_TO**, **SUBSCRIBES_TO**, **ROUTES**, **RUNS_ON**, **CONNECTS_TO**, **USES**) — the native substrate that simulation oracles traverse — rather than the derived **DEPENDS_ON** projection constructed for GNN training.

Three real-world architectures were transcribed from authentic open-source repositories using dedicated architectural adapters. The seven synthetic scenarios were produced by a parameterized topology generator: each is fully defined by a committed configuration specifying a random seed, per-entity-type counts, seven-number summaries (mean, median, standard deviation, minimum, maximum, Q_1 , Q_3) for application publish and subscribe fan-out, applications per host, library fan-in, topic payload size, and categorical distributions over the three QoS dimensions. Graph degree distributions and clustering emerge directly from these

Table 6: Generative parameters of the seven synthetic evaluation scenarios. Counts, seed and fan-out figures are read directly from the committed configurations. The modal QoS column gives the most common reliability/durability/priority value and the range of topic shares carrying them, computed from the committed topology rather than the config’s declared QoS targets, which domain-driven assignment does not always realize (Section 6.1).

Scenario	Config	Seed	Counts	Pub	Sub	Modal QoS (R/D/P)
Autonomous Vehicle (AV)	scenario_01_autonomous_vehicle	1001	80/40/4/8/20	2.5	5.0	RELIABLE/TRANSIENT_LOCAL/HIGH (85–100%)
Enterprise Pub-Sub	scenario_07_enterprise_xlarge	7007	300/120/10/40/50	3.0	4.5	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
Financial Trading	scenario_03_financial_trading	3003	60/35/5/6/18	4.0	6.0	RELIABLE/PERSISTENT/CRITICAL (51–83%)
Healthcare Integration	scenario_04_healthcare	4004	50/25/3/8/12	2.5	3.0	RELIABLE/PERSISTENT/MEDIUM (60–76%)
Hub-and-Spoke	scenario_05_hub_and_spoke	5005	70/30/2/12/25	2.0	7.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)
IoT Smart City	scenario_02_iot_smart_city	2002	200/80/6/30/10	2.0	1.5	BEST_EFFORT/VOLATILE/LOW (56–79%)
Microservices Mesh	scenario_06_microservices	6006	90/45/6/15/30	1.5	2.0	RELIABLE/TRANSIENT_LOCAL/MEDIUM (100–100%)

parameters rather than being synthetically forced. Table 6 reports the generative parameters governing each topology’s shape; complete configurations are included in the replication package.

QoS Diversity Across the Corpus. Per-topic QoS profiles are assigned via domain-keyed lookups (`get_qos_for_topic`) that take precedence over generic categorical distributions whenever a domain is designated. For three domains (**hub-and-spoke**, **microservices**, **enterprise**), that lookup table assigns an identical (reliability, durability, priority) triple across topics, while a fourth (**av**) collapses four of five entries to the same profile. Table 6’s Modal QoS column reflects this structure: those four scenarios exhibit 85%–100% topic concentration at the modal triple, compared to 51%–79% for the three domains (**finance**, **healthcare**, **iot**) featuring genuinely multi-valued profiles. Consequently, scalar coupling weight $w(t)$ has standard deviation ≈ 0.02 on a $[0, 1]$ scale in the four QoS-flat scenarios, versus 0.19–0.28 in the remaining three. We report this as a realistic corpus property rather than forcing artificial variation, preserving byte-level consistency across all cached experimental evaluations (Section 8.3).

Role of the ATM Case Study. An additional scenario, representing an Automated Teller Machine network, serves as the eighth fold for Leave-One-Scenario-Out evaluation (Section 6.3) and as the substrate for qualitative attention analysis (Section 7.3). This scenario is intentionally omitted from Table 5 and does not contribute to the 1,770 component total: it was authored as an illustrative architectural walkthrough, and its specification lacks the full seven-number statistical summaries. Because LOSO requires held-out topologies rather than parameterized ones, every LOSO evaluation reports eight folds over the ten-architecture corpus.

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than merely statically archived. Each dataset is deterministically generated from its configuration file via:

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests assert that every committed dataset regenerates *byte-identically* from its configuration and that all disk digests match the manifest. This guarantees that third parties can reproduce the exact graphs used in our experiments, rather than simply sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

We evaluate four primary predictor configurations, contrasting the proposed learned architectures with training-free topological baselines:

1. **HGL / HGL-QoS** (proposed): Relation-specific Heterogeneous Graph Transformers (Section 4), evaluated both without and with explicit 7-dimensional continuous-categorical QoS edge features.
2. **GL / GL-QoS** (proposed ablation): Homogeneous Graph Attention Networks (GAT) [45] trained on the flattened, untyped graph projection — isolating the specific performance gain conferred by relation typing.
3. **Topo-QoS** (baseline): Training-free, QoS-weighted topological centrality baseline.

4. **Topo-BL** (baseline): Training-free structural centrality combining unweighted betweenness centrality and articulation point scoring.

In addition, the out-of-distribution evaluation (Table 9) reports **RM** / $Q(v)$ (the deterministic hierarchical quality attribution model of Section 5) as a diagnostic reference baseline. RM is not fitted to rank failure impact; its inclusion demonstrates how much learned relational prediction adds over static structural attribution (Section 1.2). Furthermore, deterministic RM scoring drives every sensitivity sweep in Section 7.3, where closed-form formulations isolate parameter effects from neural training stochasticity.

6.2.1. Evaluation Substrate and Internal Validity Scope Condition

Predictors in this study operate over distinct graph views, a structural distinction critical for interpreting the empirical results. **Topo-BL**, **Topo-QoS**, **GL**, and **GL-QoS** are evaluated on the derived Application–Library **DEPENDS_ON** projection (Section 3.2). Conversely, **HGL** and **HGL-QoS** ingest the complete native typed multigraph across all five entity types. Crucially, **no predictor in either group accesses $G_{\text{structural}}$** : that raw topology is strictly reserved as the substrate for ground-truth simulation oracles (Section 4.4), a guarantee formally verified by `tests/test_independence_guarantee.py`.

The projected substrate was adopted for untyped baselines because raw publish–subscribe graphs cannot be effectively learned by homogeneous models on this corpus: Application nodes never route messages in raw pub-sub, resulting in near-zero betweenness and bridge ratios that yield degenerate, near-constant node features. A single homogeneous aggregation layer then causes all application representations to over-smooth toward identical embeddings via high-degree Topic hubs [65, 66].

Dissecting the Substrate Confound in RQ2: This architectural asymmetry introduces an important internal validity scope condition for the RQ2 comparisons in Section 7.2. When heterogeneous HGL outperforms homogeneous GL by +0.353 in out-of-distribution transfer, that performance margin conflates two distinct factors: (1) *relational edge/node parameter typing* (allowing distinct attention matrices per edge type), and (2) *multi-entity topological visibility* (direct structural access to intermediate brokers, physical hosts, and explicit topic channels that were projected out of GL’s view). While homogeneous models cannot consume raw multigraphs without over-smoothing on bipartite topic stars, future work should examine homogeneous architectures with generalized positional encodings operating over unified multigraph views to further disentangle these dimensions. Nonetheless, this architectural difference does not bias evaluation populations: all variants are scored on an identical, independently resolved Application node set (Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures in this paper use the high-contrast, colorblind-safe Okabe–Ito palette together with distinct marker, hatching and node-shape encodings, so that every distinction carried by colour is also carried by form and remains legible in monochrome.

- **Ranking Precision:** Evaluated via Spearman rank correlation (ρ) and Kendall’s rank correlation (τ) between predicted component rankings and ground-truth simulated impact $I^*(v)$ from the primary oracle (Section 4.3).
- **Critical-Set Identification:** Measured via $F_1@K$, Precision@ K , and Recall@ K for top- K critical components, where $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$. Because predicted and ground-truth sets both contain exactly K elements, Precision, Recall, and F_1 coincide identically as the top- K set overlap.
- **Statistical Significance:** Assessed through paired Wilcoxon signed-rank tests [67] ($p < 0.05$) and non-parametric bootstrap 95% confidence intervals ($B = 2,000$) [68, 69].

6.3.1. Evaluation Population

Every predictor within a given evaluation table is scored on an identical node population, resolved strictly from scenario topology and simulation ground truth — never from any model’s predictions. Unless otherwise noted, this population is the **Application** set (V_{app}). This aligns with the framework’s primary objective (forecasting application-layer cascading failures) and ensures a fair common denominator across both typed

and untyped predictors. Pooling node types into a single global ranking conflates distinct base rates and impact distributions, shifting the resulting rank correlation outside the envelope of per-type correlations (Section 7.3). We therefore report stratified, single-population metrics throughout and explicitly identify any pooled figures.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits pinned deterministically by node identity within each scenario, evaluated across five random seeds {42, 123, 456, 789, 2024}.
- **Inductive Leave-One-Scenario-Out (LOSO):** Models are trained on seven synthetic scenarios and evaluated zero-shot on the held-out scenario across eight folds (including the ATM topology), testing zero-shot generalization across distinct architectural domains.
- **Real-World Architectural Transfer:** Evaluating models trained on synthetic corpora zero-shot on authentic open-source distributed systems without fine-tuning.

7. Results and Empirical Analysis

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 7 presents the in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all seven synthetic scenarios.

Table 7: In-distribution held-out Spearman rank correlation (ρ) against $I^*(v)$ (seed means over 5 seeds; n is held-out Application count). With held-out counts of 10–60, single-scenario ρ values carry wide uncertainty and should be read alongside the paired tests of Table 8 rather than individually.

Scenario	n	Topo-BL	Topo-QoS	GL	GL-QoS	HGL	HGL-QoS
AV System	16	0.283	0.701	0.764	0.678	0.789	0.649
Enterprise	60	0.420	0.789	0.871	0.483	0.880	0.891
Financial Trading	12	0.289	0.700	0.645	0.539	0.854	0.770
Healthcare	10	−0.101	0.772	0.623	0.463	0.652	0.645
Hub-and-Spoke	14	0.234	0.359	0.547	0.054	0.534	0.297
IoT Smart City	40	−0.063	0.073	0.289	0.291	0.881	0.842
Microservices	18	0.401	0.707	0.525	0.564	0.483	0.476
Mean	—	0.209	0.586	0.609	0.439	0.725	0.653

Table 8: Paired Wilcoxon signed-rank tests across scenarios ($n = 7$, two-sided).

Comparison	$\Delta\rho$	Won	Wilcoxon W	p -value	Significance
HGL vs. Topo-BL	+0.516	7/7	0.0	0.0156	Statistically Significant ($p < 0.05$)
HGL vs. GL-QoS	+0.286	6/7	1.0	0.0312	Statistically Significant ($p < 0.05$)
HGL vs. Topo-QoS	+0.139	5/7	9.0	0.469	Not significant
GL vs. Topo-QoS	+0.023	4/7	10.0	0.578	Not significant
HGL vs. GL	+0.116	5/7	7.0	0.297	Not significant
HGL-QoS vs. HGL	−0.072	1/7	3.0	0.078	Not significant (marginal; HGL-QoS trails in-distribution)

7.1.1. Out-of-Distribution (LOSO) Generalization

In inductive Leave-One-Scenario-Out (LOSO) cross-validation, models are evaluated on their capacity to predict cascading criticality over completely unseen system topologies:

Eight LOSO folds are reported, comprising the seven synthetic scenarios and the ATM case study from Section 7.3. In each fold, one scenario is held out for zero-shot testing while the model is trained exclusively

Table 9: Inductive Leave-One-Scenario-Out (LOSO) evaluation, Application population, eight folds. Rows are grouped by role: training-free structural baselines, proposed learned predictors, and the RM/ $Q(v)$ diagnostic reference of Section 1.2.

Predictor / Reference	Mean LOSO ρ	Std ρ	Critical-Set $F_1@K$	Requires Training
<i>Training-free structural baselines</i>				
Topo-BL	0.301	0.126	0.363	No
Topo-QoS	0.571	0.181	0.380	No
<i>Learned predictors</i>				
GL (Homogeneous)	0.086	0.122	0.237	Yes
GL-QoS (Homogeneous)	0.363	0.089	0.341	Yes
HGL (Typed Heterogeneous)	0.439	0.145	0.327	Yes
HGL-QoS (Typed + QoS)	0.608	0.143	0.414	Yes
<i>Diagnostic reference — not a ranking model</i>				
RM / $Q(v)$	0.195	0.130	0.327	No

on the remaining seven. All variants are evaluated on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests are conducted across the eight folds.

Key Insights for RQ1:

- The typed predictor is the best learned configuration overall.** HGL-QoS leads every other learned variant on both metrics ($\rho = 0.608$, $F_1@K = 0.414$), with a significant ranking margin over both homogeneous GNNs (+0.246 over GL-QoS and +0.522 over GL, 8/8 folds, $p = 0.0078$ in both cases). Section 7.2 states the substrate caveat that qualifies this comparison.
- Critical-set identification separates learned models from each other, but not from the untrained QoS baseline.** HGL-QoS improves $F_1@K$ over GL by +0.177. Against Topo-QoS the margin is only +0.034 (0.414 vs. 0.380); we report no significance test for it, and it should not be read as an established advantage. With K between 2 and 12 components per fold, a single component changing rank in one fold moves the mean by more than this margin, and the gap is smaller than the oracle’s own seed-to-seed critical-set reproducibility (Section 8.3).
- QoS encoding carries the typed model out of distribution.** HGL-QoS leads HGL by $\Delta\rho = +0.169$, winning 7 of 8 folds ($p = 0.0156$) — a substantial, consistent advantage under distribution shift.
- Ranking Boundary:** Against *Topo-QoS*, HGL-QoS’s margin is +0.037, won in 5 of 8 folds ($p = 0.64$, not statistically significant). On out-of-distribution ranking, heterogeneous graph learning matches rather than outperforms a well-constructed QoS baseline, and the critical-set margin above is untested. The case for training a model therefore rests on the qualitative capabilities the baseline cannot supply — actionable explanations, per-relationship edge criticality, and multi-task quality attributions — which this study describes but does not yet quantify (Section 8.4).
- The explanation layer is weakly predictive, not noise.** RM/ $Q(v)$ achieves $\rho = 0.195$ under distribution shift — outperforming untyped GL (0.086) while providing interpretable architectural diagnostics (Section 5).

7.2. RQ2: Value of Typed Heterogeneity

To evaluate the specific contribution of node and edge typing, we contrast relation-specific HGL against homogeneous GL:

- In-Distribution:** Heterogeneous HGL leads homogeneous GL by $\Delta\rho = +0.116$ (0.725 vs. 0.609; $p = 0.297$, 5/7 scenarios won). On familiar topologies, homogeneous GNNs partially approximate typing via structural degree signatures.
- Out-of-Distribution (LOSO):** The gap widens sharply under distribution shift. Heterogeneous HGL outperforms homogeneous GL by +0.353 ($\rho = 0.439$ vs. 0.086), winning *all eight* folds (paired Wilcoxon,

Software-as-a-Graph (SaG) Pre-Deployment Reliability Evaluation — application population

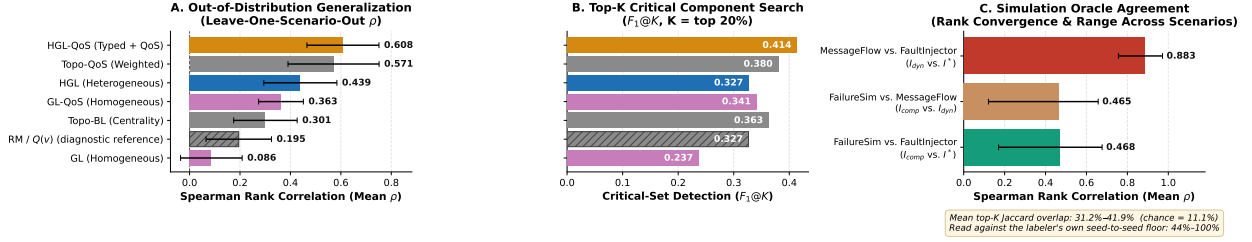


Figure 3: Results at a glance, evaluated on the Application population. (A) Out-of-distribution rank correlation per variant across eight LOSO folds (whiskers denote σ). (B) Critical-set identification at $K = 20\%$. (C) Pairwise agreement across the three simulation oracles.

$p = 0.0078$). Similarly, HGL-QoS outperforms GL-QoS by +0.246 (0.608 vs. 0.363, $p = 0.0078$). Without relation typing, homogeneous models degrade below unweighted structural baselines (0.086 vs. 0.301).

Scope Condition and Confound Dissection. As detailed in Section 6.2.1, GL and HGL are evaluated on distinct graph views: GL/GL-QoS operate on the Application–Library DEPENDS_ON projection (preventing structural collapse on raw graphs), whereas HGL/HGL-QoS ingest the complete native multigraph. Consequently, HGL’s out-of-distribution margin reflects both relation-specific attention parameterization and direct topological visibility into the multi-entity infrastructure layer (brokers, topics, and host nodes). We therefore report this result as a comparison between typed learning on the full multigraph and untyped learning on a projected view, not as an isolated measurement of typing. The control that would isolate typing — an untyped architecture over the identical multigraph with node-type indicators supplied as input features — is not run here and is the principal outstanding validation of our central claim.

7.2.1. Empirical Edge-Removal Analysis

We further evaluated edge criticality by simulating the removal of candidate communication channels while keeping endpoint components operational. Across 50 candidate bridge edges in the `av_system` topology, 46 edges produced zero downstream cascade impact, while the 4 edges with non-zero impact were direct communication channels (PUBLISHES_TO / SUBSCRIBES_TO). This confirms that individual communication links are largely redundant, whereas component outages and shared library dependencies induce disproportionate systemic disruption.

7.3. RQ3: Ablations and Sensitivity Analysis

Role of ρ in Sensitivity Sweeps. The parameter sweeps below utilize RM’s rank correlation against $I^*(v)$ strictly as a *sensitivity probe* to quantify parameter leverage relative to between-model margins.

7.3.1. QoS Feature Encoding

Explicitly encoding continuous QoS attributes (HGL-QoS) trades minor in-distribution accuracy for substantial out-of-distribution robustness:

Table 10: HGL-QoS against HGL under in-distribution and LOSO protocols (Application population).

Protocol	HGL	HGL-QoS	$\Delta\rho$	Folds won	Wilcoxon p
In-distribution (Table 7)	0.725	0.653	−0.072	1/7	0.078 (n.s.)
Inductive LOSO (Table 9)	0.439	0.608	+0.169	7/8	0.016

In-distribution, HGL-QoS trails base HGL slightly ($\Delta\rho = -0.072$, not statistically significant), concentrated in two atypical topologies (Hub-and-Spoke and AV). Under LOSO, this relationship reverses decisively:

HGL-QoS leads by +0.169 ($p = 0.016$, 7/8 folds won). While structural degree signatures do not transfer across unseen architectures, declared QoS contracts preserve identical semantics (**RELIABLE** and **PERSISTENT**) everywhere.

7.3.2. Topic-Weight Coefficients (β, α, ψ)

We swept (β, α, ψ) over a grid spanning the entire simplex to evaluate sensitivity to topic coupling weights:

Table 11: Sensitivity of topic-weight ordering and downstream rank correlation against $I^*(v)$ to coefficients (β, α, ψ) (Application population, mean over 7 scenarios, 375 topics).

(β, α, ψ)	ρ of $w(t)$ vs. shipped	Topo-QoS ρ	RM ρ
(0.75, 0.15, 0.10) <i>shipped</i>	1.000	0.604	0.267
(0.90, 0.05, 0.05)	0.966	0.600	0.273
(0.85, 0.15, 0.00)	0.950	0.608	0.268
(1.00, 0.00, 0.00)	0.852	0.618	0.268
(0.60, 0.20, 0.20)	0.970	0.604	0.261
(0.50, 0.25, 0.25)	0.942	0.603	0.259
$(\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ <i>uniform</i>	0.870	0.608	0.256
Spread over grid	—	0.018	0.017

Topic-weight coefficients are not load-bearing: the ordering of $w(t)$ never falls below $\rho = 0.852$ against the default, and downstream rank correlations vary by at most 0.018 (Topo-QoS) and 0.017 (RM).

7.3.3. Intra-Dimension AHP Weight Shrinkage

We evaluated the sensitivity of the RM attribution baseline across shrinkage parameter $\lambda \in [0, 1]$, blending internal term weights from a uniform prior ($\lambda = 0$) to raw AHP judgment ($\lambda = 1$):

Table 12: Sensitivity of RM rank correlation against $I^*(v)$ to AHP shrinkage parameter λ (Application population, mean over 7 scenarios).

λ Setting	0.00 (Uniform)	0.50	0.70 (Default)	0.80	1.00 (Raw AHP)
Mean Rank Correlation (ρ)	0.348	0.291	0.267	0.256	0.232

Rank correlation decreases monotonically as weights transition from uniform toward raw AHP ($\rho = 0.348 \rightarrow 0.232$, Figure 4). Elicited AHP weights provide transparent, auditable domain attribution rather than optimizing rank correlation. We retain $\lambda = 0.70$ on that basis.

7.3.4. Joint Sensitivity Across All Ten Weight Constants

To assess interactions, we swept all ten weight constants jointly across six scenarios using Morris elementary-effects screening [70, 71] (a computationally efficient alternative to variance-based global sensitivity analysis [72, 73]):

Morris screening confirms that λ and r_α are the primary drivers, while topic and QoS sub-weights reside in a modest band ($\mu^* \in [0.012, 0.019]$). Dirichlet simplex sampling over 100 draws confirms tight stability: mean $\rho = 0.243$ (standard deviation 0.006, 90% interval [0.231, 0.253], mean top-20% Jaccard 0.825).

7.3.5. Convergent Validity Over Simulation Oracles

We evaluated inter-oracle agreement across $I^*(v)$ (**FaultInjector**), $I_{\text{comp}}(v)$ (**FailureSimulator**), and $I_{\text{dyn}}(v)$ (**MessageFlowSimulator**) over seven scenarios: Ordering converges strongly between the queue-flow simulator and topological cascade oracle ($\rho = 0.883$). Top- K set agreement is more conservative (Jaccard 0.42), bounded by discrete threshold sensitivity in non-linear cascades.

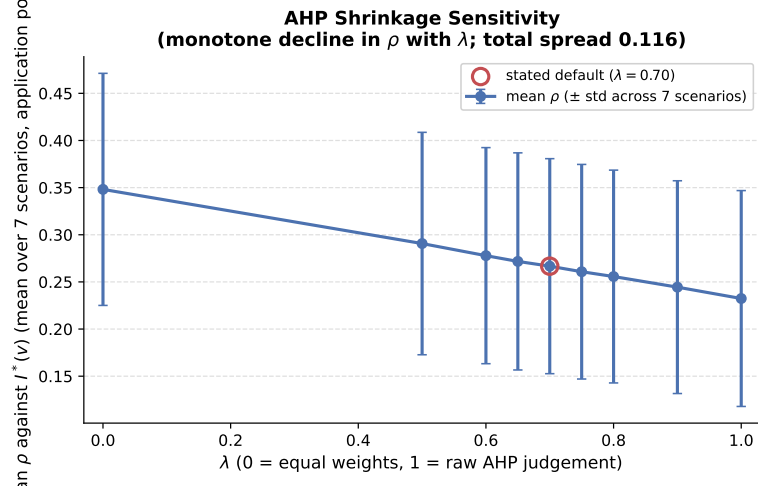


Figure 4: Sensitivity of RM composite rank correlation against $I^*(v)$ across AHP shrinkage λ (Application population, mean over 7 scenarios).

Table 13: Morris elementary-effects screening [70, 71] across ten weight constants, ranked by influence (μ^*) on mean ρ (6 scenarios, 10 trajectories, 110 evaluations).

Factor	μ^*	σ
λ (AHP shrinkage)	0.124	0.077
r_α	0.096	0.035
α	0.019	0.034
w_{rel}	0.019	0.018
β	0.017	0.023
w_{dur}	0.014	0.017
w_{prio}	0.013	0.018
ψ	0.012	0.015
p (power-mean exponent)	0.005	0.005
γ (fan-out)	0.001	0.002

Table 14: Inter-oracle agreement across simulation paradigms (chance top- K Jaccard is 0.111).

Oracle pair	Mean ρ (range)	Mean τ	Jaccard@ K	Tie-robust
I_{dyn} vs. I^*	0.883 (0.756–0.972)	0.745	0.424	0.419
I_{comp} vs. I^*	0.468 (0.171–0.677)	0.349	0.307	0.312
I_{comp} vs. I_{dyn}	0.465 (0.121–0.658)	0.348	0.313	0.313

7.3.6. Domain-Specific Weighting and Threshold Sensitivity

Sweeping composite reliability weight $w_R \in [0, 1]$ moves mean ρ by only 0.024 ($0.341 \rightarrow 0.365$), with mean Kendall correlation $\tau = 0.974$ between domain and static rankings. Sweeping cascade propagation thresholds revealed higher sensitivity ($\Delta\rho = 0.102$), with ranking performance plateauing above 0.35.

7.3.7. Availability Metric Recalibration

In initial formulations, hard articulation-point gating caused Availability $A(v)$ to collapse to a constant multiple of $w(v)$ in six of eight scenarios (the pre-recalibration formulation weighted $w(v)$ at 0.05; the shipped formulation of Equation (16) weights it at 0.0878) where applications do not form cut vertices. Replacing the hard cut-vertex gate with a continuous redundancy-deficit metric (fractional increase in average shortest paths upon removal) widened σ_A to $[0.0254, 0.0530]$ and restored structural discriminability.

7.3.8. Anti-Pattern Detection and Node-Type Stratification

Validating our rule-based anti-pattern catalog against $I_{\text{comp}}(v)$ yielded mean precision 0.239 and recall 0.900 ($F_1 = 0.3781$), reflecting a high-recall CI/CD safety posture. Path-enumeration detector DEEP_PIPELINE timed out on larger graphs, identifying a known scalability limitation of exhaustive search.

Stratification vs. Pooling: Stratified RM rank correlations are $\rho = 0.503$ (Application), 0.395 (Broker), and 0.142 (Node), while pooled correlation across all types is $\rho = 0.028$. Pooling node types triggers Simpson’s paradox by mixing distinct base rates. Classifying components within node types changes the criticality tier of 62.8% of components (with 19.0% crossing the critical boundary), confirming that evaluation and gating must operate strictly per-type.

7.3.9. HGT Attention Weight Analysis

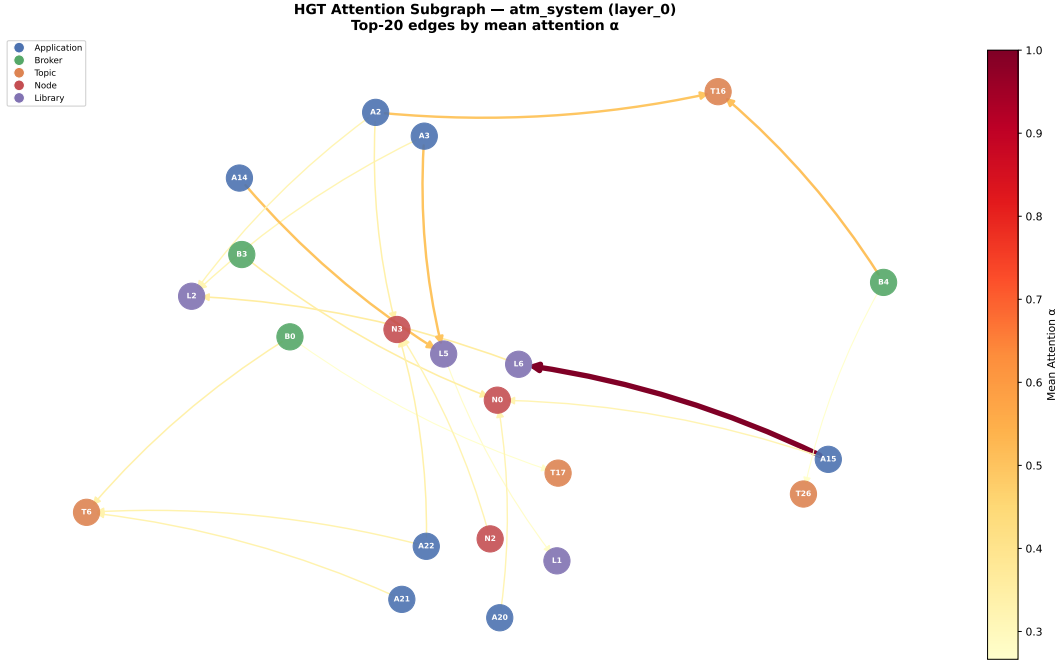


Figure 5: Relational attention extracted from the trained HGT on the ATM case study. Peak attention focuses on **USES** (Application $\rightarrow$ Library) and **ROUTES** (Broker $\rightarrow$ Topic) channels.

Extracted attention weights (Figure 5) show that the HGT places maximum attention ($\alpha_{uv} = 1.00$) on **USES** edges (shared libraries) and high attention (≈ 0.50) on **ROUTES** edges (broker routing), mirroring the shared-library blast-radius and broker-bottleneck pathways rather than sequential pub-sub hops.

7.4. RQ4: Real-World Distributed Architecture Validation

We evaluated SaG zero-shot on three open-source distributed systems transcribed from their public repositories by dedicated architectural adapters. We note the scope of this evaluation before reporting it: Online Boutique is a vendor demonstration application and Train-Ticket an academic benchmark, so these are reference implementations rather than production deployments; each is an abstraction of its upstream repository rather than a complete transcription; and their labels come from the same simulation oracle used throughout, so what follows tests topological transfer, not agreement with observed field failures.

Table 15: Zero-shot transfer evaluation on open-source reference systems (held-out, without fine-tuning). Ground truth is simulator-derived, as elsewhere in this study; these are not observed production incidents.

Real-World Architecture	$ V $	$ V_{\text{app}} $	Spearman ρ	Kendall τ	App $F_1@K$	Pooled $F_1@K$	Impactful Apps	Gain vs. Deg
Autoware.universe (ROS 2)	75	32	0.688 $\pm$ 0.009	0.517	0.333	0.800	19 / 32	+0.360
Cloud Microservices Mesh	60	22	0.778 $\pm$ 0.001	0.639	0.500	1.000	8 / 22	+0.014
Train-Ticket Booking Mesh	90	41	0.759 $\pm$ 0.001	0.605	0.625	1.000	14 / 41	+0.264

Key Insights for RQ4:

- 1. Strong Real-World Zero-Shot Transfer:** Trained exclusively on synthetic scenarios, SaG achieves high zero-shot rank correlation on Cloud Microservices ($\rho = 0.778$), Train-Ticket ($\rho = 0.759$), and Autoware.universe ($\rho = 0.688$), indicating that relation-specific graph representations transfer to architectures outside the generator’s distribution. Because the comparison in this table is against degree centrality alone, it does not establish transfer superiority over the stronger Topo-QoS baseline of Table 9.
- 2. Stratified vs. Pooled Critical Set Identification:** On the stratified Application population (V_{app} with $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$), SaG identifies the top-20% critical services with $F_1@K = 0.333$ in Autoware, 0.500 in Cloud Microservices, and 0.625 in Train-Ticket. When evaluated across the pooled multigraph ($K = \text{round}(0.20 \cdot |V|)$), the pooled $F_1@K$ reaches 0.800, 1.000, and 1.000, respectively. This disparity highlights the base-rate phenomenon discussed in Section 6.3: unmeasured passive infrastructure entities (Topics, Nodes, Libraries) exhibit constant zero failure impact under process crash simulation. While the pooled metric confirms that SaG cleanly separates critical active services from inert infrastructure, reporting stratified V_{app} metrics provides the more rigorous, non-inflated evaluation.
- 3. Predictive Advantage over Structural Heuristics:** The “Gain vs. Deg” column reports SaG’s ranking margin over an unweighted degree centrality baseline (+0.360 on Autoware, +0.264 on Train-Ticket, +0.014 on Cloud Microservices). Heterogeneous message passing captures multi-hop topic-broker cascades that degree and local connectivity metrics fail to resolve.

7.5. RQ5: Analysis Cost, Computational Sustainability, and CI/CD Feasibility

RQ5 quantifies computational overhead and sustainability during CI/CD evaluation:

Table 16: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs).

$ V $	$ E $	Analyse (s)	Graph→tensor (s)	HGT forward (ms)	Analyse : forward
249	1,127	0.27	0.011	22.5	12×
499	2,402	0.95	0.022	15.7	61×
999	6,422	4.74	0.055	36.7	129×
1,998	19,301	23.83	0.153	43.7	545×

The Neural Model is the Cheapest Stage, but Not the Whole Cost. For a 2,000-component topology, HGT forward inference requires only 43.7ms, while classical structural analysis takes 23.8s (545× slower). This ratio is a statement about where the cost sits inside the pipeline, not about the cost of evaluating an architecture: indices 0–17 of every node feature vector (Section 3.4) are betweenness, closeness, reverse

PageRank, articulation and bridge scores produced by that same analysis stage, so the forward pass cannot run without it. End-to-end pre-deployment evaluation of a previously unseen 2,000-component architecture therefore costs approximately 24 s, of which the neural model is 0.2%. The 43.7 ms figure is the marginal cost of re-scoring a graph that has already been analysed. The computational bottleneck resides entirely in deterministic graph algorithms ($O(|V|^2 + |V||E|)$). Computational overhead scales primarily with edge density: the 520-component Enterprise mesh requires 27.2 s due to 3,245 structural edges, whereas a sparser 999-component system takes only 4.7 s. Across all corpus scenarios, the complete quality gate (structural analysis plus 18 anti-pattern detectors) runs in 0.02 s to 27.4 s, well within standard pull-request CI/CD budgets.

Computational Efficiency. The quantity we measured is wall-clock time: the complete gate runs in 0.02–27.4 s per architecture, against multi-seed discrete-event simulation sweeps that are substantially more expensive on the same corpus. We deliberately do not convert this into an energy or carbon figure. No physical power counters were instrumented in this study (Section 8.2), the mapping from CPU-seconds to joules depends on hardware and utilisation we did not control, and a latency ratio is not an energy ratio. What the measurements support is the narrower claim that pre-deployment architectural gating fits inside a pull-request CI/CD budget without a simulation cluster.

8. Discussion, Threats to Validity, and Conclusion

8.1. Discussion and Practical Implications

Our empirical findings provide clear, actionable guidance for software architects and site reliability engineers:

- **Role of the Predictive Pathway:** Heterogeneous graph learning proves superior in three critical aspects:
 1. *Zero-Shot Inductive Generalization:* It is the most effective *learned* model on unfamiliar architectures ($\rho = 0.608$ out-of-distribution), where untyped GNNs over the projected view fail to transfer ($\rho = 0.086$), subject to the substrate caveat of Section 7.2.
 2. *Shortlist Identification:* It attains the highest critical-set score of any variant we evaluated ($F_1@K = 0.414$), though its margin over the training-free QoS baseline is small and untested (Section 7.1).
 3. *Low Marginal Cost:* Re-scoring an analysed graph costs 43.7 ms, and the complete gate including feature computation runs in 0.02–27.4 s — fast enough for pull-request gating rather than deferred nightly batch sweeps (Section 7.5).

Beyond scalar rankings, HGT provides relation-specific attention over the channels driving cascade spread, per-relationship edge criticality, and multi-task quality attributions that degree centralities cannot produce.

- **Boundary Conditions and When Not to Train:** The honest comparison is against a training-free QoS-weighted centrality baseline (**Topo-QoS**), where the out-of-distribution ranking margin is +0.037 and not statistically significant ($p = 0.64$, Table 9). For engineering teams requiring only a component ranking on architectures similar to those already characterized, **Topo-QoS** is recommended: it requires no training, corpus, or model checkpoints. Graph learning is justified when explanatory attributions, per-relationship criticalities, or inductive zero-shot transfer across structurally distinct architectures are required. Between the two typed variants, HGL provides higher in-distribution fidelity, while HGL-QoS provides essential robustness under distribution shift (Section 7.3).
- **Role of the Explanation Layer:** The deterministic RM profile renders ranked components actionable. Specifically, distinguishing between services critical due to single-point-of-failure exposure (Availability) versus wide error propagation (Fault Tolerance) determines whether architects should deploy load-balanced replicas or configure circuit-breaker policies. This causal diagnosis is not captured by either the predictive GNN or the simulation oracle.

This operational division mirrors the architecture in Figure 1: the explanation layer (RM/ $Q(v)$) and the predictive pathway (Topo-BL/Topo-QoS/GL/HGL) answer distinct questions from the same unified graph. Consequently, Table 9 and Figure 3 report RM alongside the ranking predictors as an interpretable diagnostic reference rather than a competing ranking model.

8.2. Performance and Computational Sustainability Implications

The cost profile in Section 7.5 reverses the conventional intuition about deep learning pipelines: within this pipeline the neural model is the cheapest stage, and its margin over the deterministic stages widens monotonically with system scale (43.7 ms inference vs. 23.8 s for structural analysis at 2,000 components; $12\times$ at 249 nodes, $545\times$ at 1,998 nodes). The analysis stage is a prerequisite rather than an alternative, so the deployable figure is the combined 0.02–27.4 s gate. Two software engineering consequences follow:

1. **Green CI/CD Quality Gating vs. Simulation Waste:** In cloud-native continuous integration, evaluating architectural resilience via continuous chaos engineering or multi-seed discrete-event simulation is computationally prohibitive. A multi-seed dynamic fault injection and message-flow sweep over a 2,000-component topology is substantially more expensive than the 0.02–27.4 s static gate measured here, and requires cluster capacity the gate does not. We state this as a difference in compute time, which is what we measured; we do not convert it into joules or CO₂, because we instrumented no power counters and the conversion depends on hardware and utilisation outside our control. The engineering consequence that follows from the measurement alone is that per-commit architectural verification is affordable without a simulation cluster.
2. **Operational Sustainability via Avoided Cascade Energy:** Beyond pre-deployment testing budgets, the primary sustainability dividend of SaG lies in *avoided operational compute* in production. When an architectural single point of failure or unbuffered dependency collapses under load, the resulting cascade triggers runaway retry storms, exponential backoff polling, container restart thrashing, and database connection pool starvation. Each of these consumes CPU, memory and network capacity without completing useful work: re-initializing crashed services, unthrottled downstream retransmission, and worker threads blocked on saturated queue backpressure. Preventing a cascade at design time avoids that expenditure entirely, which is a stronger sustainability argument than any saving on the analysis itself.
We deliberately stop short of quantifying it. Doing so would require a per-component energy model, an incident-duration distribution, and a causal estimate of how many incidents the gate actually prevents — none of which we measured, and the last of which cannot be established without a longitudinal deployment study. We therefore record avoided cascade energy as the motivating hypothesis for this line of work rather than as a result of it, and we make no numerical claim about its magnitude.

Empirical Boundary and Scope Disclosure: We state the empirical boundary explicitly: *no direct physical hardware power measurements were conducted in this study*. All reported runtimes are single-threaded CPU wall-clock execution times. We did not instrument physical package energy via Intel RAPL or GPU-side power via NVIDIA NVML, nor did we measure the exact physical energy consumption of production outages. Consequently, operational sustainability claims are structurally reasoned and conditional on incident prevention. Direct hardware power instrumentation remains an important objective for future work (Section 8.4).

8.3. Threats to Validity

- **Construct Validity:** Our ground truth is generated via discrete-event cascade simulation on structural models rather than observing live production outages. To characterize construct divergence, we evaluated the three reliability-facing simulation engines (**FaultInjector**, **FailureSimulator**, **MessageFlowSimulator**) of our four-oracle taxonomy (Section 4.3). On *ordering*, the evidence converges strongly: the behavioural queue-flow simulator and the topological cascade oracle agree at $\rho = 0.883$ (never below 0.756 across scenarios), confirming that $I^*(v)$ tracks dynamic service disruption reach rather than purely static connectivity. However, queue simulation deltas are unweighted by

message priority and only partially sensitive to delivery policies. On *critical-set membership*, agreement is lower: top- K Jaccard sits between 0.31 and 0.42 (against 0.111 by chance). This is not an artifact of tie-breaking (≤ 0.005 shift under tie-robust cuts), but reflects intrinsic non-linear cascade threshold sensitivity and oracle self-agreement noise (oracles achieve only 0.44 Jaccard against their own re-runs in worst-case scenarios). A further 30–47% of components per scenario carry no simulated ground truth because the cascade model cannot express direct Topic or Node failure. The fourth oracle, $I_M(v)$ (**ChangePropagationSimulator**), targets Maintainability and traverses the same derived dependency topology from which $M(v)$ is scored; agreement between them serves as an internal consistency check rather than independent validation. Settling this requires an external referent independent of topology, such as version-control code churn or historical co-change coupling, left to future work.

- **Internal Validity and Substrate Confound:** Potential feature leakage is prevented by strict graph view separation: predictors operate exclusively on G_{analysis} , whereas ground-truth simulators operate on $G_{\text{structural}}$. We identified and resolved a normalization defect in the code-quality scoring pipeline: a min-max normalization helper previously assigned a maximal penalty to zero-variance populations, penalizing unanalyzed libraries lacking source-level `code_metrics`. Scoring undifferentiated populations with zero penalty corrected library Maintainability tiers while leaving Application rank correlations unaffected (Table 15: $\Delta\rho \in \{-0.003, 0.000, 0.000\}$). In addition, four of the seven synthetic scenarios feature domain-lookup QoS tables that assign identical profiles across topics ($w(t)$ standard deviation ≈ 0.02). **Topo-QoS** nevertheless gains $+0.305\rho$ over **Topo-BL** in those scenarios (Table 7), a gain resulting from uniform edge rescaling rather than QoS diversity. This corpus characteristic limits what the QoS ablations of Section 7.3 can establish: on four of seven scenarios there is almost no QoS variation for the encoding to exploit.

Furthermore, as noted in Section 6.2.1, the RQ2 typing comparison between HGL and GL involves an experimental substrate asymmetry: GL operates on the projected subgraph to avoid over-smoothing on bipartite topic stars, whereas HGL operates on the full multigraph. Consequently, HGL’s out-of-distribution margin reflects both relation-specific attention parameterization and multi-entity topological visibility into intermediate broker and host infrastructure.

- **External Validity and Real-World Metric Stratification:** While our evaluation spans ten architectures across six domains, including three authentic open-source distributed systems (ROS 2 Autoware, Cloud Microservices, Train-Ticket), future work should evaluate larger enterprise deployments with hundreds of microservices. Crucially, as shown in Table 15, evaluating top- K critical sets across the pooled multigraph yields elevated $F_1@K$ scores (0.800–1.000) because unmeasured infrastructure entities carry constant zero failure labels. Reporting stratified metrics on active application services ($F_1@K = 0.333, 0.500, 0.625$) provides the true, non-inflated operational baseline.
- **Sustainability Claims Are Unmeasured:** The sustainability arguments in Section 8.2 rest on wall-clock latency reductions and avoided operational recovery compute, neither of which was instrumented using physical watt meters. No direct physical energy or carbon figures appear in this study, and none should be inferred; empirical power instrumentation is reserved for future hardware testbeds.
- **Conclusion Validity:** Given the heavy-tailed, non-normal distribution of cascading failure impacts, all statistical comparisons utilize non-parametric rank correlation (Spearman ρ , Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests. Two critical hazards that altered conclusions in this study deserve explicit statement:

1. *Node-Type Aggregation (Simpson’s Paradox):* Pooling rank correlations across different node types yields $\rho = 0.028$, falling completely outside the range of per-type correlations ($\rho \in [0.14, 0.50]$). Pooling placed the RM baseline at $\rho = -0.014$ and untyped GL at 0.381, whereas stratified evaluation on the Application population places them at +0.195 and 0.086, respectively, reversing their relative standing. Consequently, all reported metrics are stratified on a single stated population (V_{app} , Section 6.3).

2. *Substrate Conflation:* Early RM ablations scored $Q(v)$ from features restricted to the Application–Library DEPENDS_ON projection, where cut-vertex Availability terms vanished, producing false negative rank correlations. Computing features through the full multigraph pipeline resolved this issue.

8.4. Limitations and Future Work

1. **Distributed AI and LLM Serving Topologies:** Modern AI infrastructure relies on distributed LLM serving systems (e.g., vLLM, Triton, DeepSpeed) characterized by complex tensor and pipeline parallelism across GPU nodes, dynamic KV-cache routing, and disaggregated prefill-decode architectures. A failure or straggler node in a pipeline-parallel ring induces severe head-of-line blocking and massive GPU idle power dissipation. Extending the SaG multigraph schema to model distributed model serving topologies (nodes representing GPU workers, edges encoding tensor communication channels and KV-cache transfer fabrics) offers a promising avenue to forecast and mitigate bottlenecks in AI serving clusters.
2. **Physical Hardware Power Counter Instrumentation:** Validating the sustainability thesis through empirical power measurements using running package counters (Intel RAPL, AMD RAPL, NVIDIA NVML) and IPMI baseboard sensors on physical Kubernetes clusters during live fault-injection and chaos experiments.
3. **Hardware-in-the-Loop (HIL) Cyber-Physical Validation:** Validating SaG’s predictions against real-time physical fault-injection testbeds in cyber-physical environments, such as autonomous driving compute boxes running ROS 2 with CAN-bus hardware interfaces.
4. **Automated Architectural Refactoring and Self-Healing:** Extending SaG from predictive analysis to prescriptive synthesis—automatically generating pull requests that reconfigure QoS policies, insert circuit-breakers, and add redundant broker pathways to eliminate single points of failure.

8.5. Conclusion

This work introduced **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that bridges the Architecture–Code Gap by combining a relation-specific Heterogeneous Graph Transformer (HGT) for failure-impact forecasting with an interpretable ISO/IEC 25010 Reliability–Maintainability explanation layer. SaG addresses three concerns of modern software engineering: **performance** (a complete pre-deployment gate in 0.02–27.4s, of which the model is 43.7ms, and identification of queue saturation bottlenecks), **reliability** (inductive cross-topology failure blast-radius forecasting on unseen architectures), and **computational cost** (a static gate that replaces multi-seed simulation sweeps and needs no cluster).

Our empirical results across synthetic systems and three open-source reference systems (ROS 2 Autoware, Cloud Microservices, Train-Ticket) show that typed heterogeneous learning over the full multigraph generalizes out of distribution, reaching $\rho = 0.608$ where untyped learning over a projected view collapses to $\rho = 0.086$ ($p = 0.0078$); isolating how much of that margin is attributable to typing alone, rather than to the additional entity types the typed model can see, requires a control we identify but do not run. We are equally explicit about the boundary: against an unparameterized QoS-weighted centrality baseline (**Topo-QoS**), out-of-distribution ranking is matched rather than beaten ($\Delta\rho = +0.037$, $p = 0.64$), and the critical-set margin ($F_1@K = 0.414$ vs. 0.380) is untested and within oracle seed noise. On present evidence a team needing only a ranking should use the baseline; the case for the learned model rests on the relational attention, per-relationship criticality and standards-grounded attribution the baseline cannot produce — specified and shown qualitatively here, and quantifying them is the natural next step. SaG’s contribution is therefore the typed substrate on which both prediction and explanation can be posed at all, with an honest account of what it currently buys.

CRediT authorship contribution statement

FIRST-NAME LAST-NAME: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **FIRST-NAME LAST-NAME:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The complete replication package—including the seven synthetic scenario datasets and the configurations that generate them, the topology generator, the four simulation harnesses, the real-world architecture adapters, trained model checkpoints, and all analysis scripts behind the reported tables and figures—is openly available at the project repository. The synthetic corpus is regenerable: each dataset carries its random seed and SHA-256 cryptographic digest in a committed manifest, with automated regression tests asserting byte-identical regeneration from configuration files (Section 6.1.1). Every table and figure in this paper is produced deterministically from committed artifacts by reproducible scripts; none of the reported values is transcribed manually.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used AI-assisted language tools to check grammar, improve readability, and support LaTeX typesetting and structure. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [9] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [10] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).

- [11] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS), 1998, pp. 68–78.
- [12] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3rd Edition, Addison-Wesley, 2012.
- [13] W. Cunningham, The WyCash portfolio management system, in: Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA), 1992, pp. 29–30.
- [14] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR), 2009, pp. 255–258.
- [15] SonarSource, Clean as you code, SonarQube documentation (2024).
- [16] T. J. McCabe, A complexity measure, IEEE Transactions on Software Engineering SE-2 (4) (1976) 308–320.
- [17] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, IEEE Transactions on Software Engineering 20 (6) (1994) 476–493.
- [18] N. Fenton, J. Bieman, Software Metrics: A Rigorous and Practical Approach, 3rd Edition, CRC Press, 2014.
- [19] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, IEEE Software 33 (3) (2016) 35–41.
- [20] L. C. Freeman, A set of measures of centrality based on betweenness, Sociometry 40 (1) (1977) 35–41.
- [21] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, Computer Networks and ISDN Systems 30 (1–7) (1998) 107–117.
- [22] U. Brandes, A faster algorithm for betweenness centrality, Journal of Mathematical Sociology 25 (2) (2001) 163–177.
- [23] M. E. J. Newman, Networks: An Introduction, Oxford University Press, 2010.
- [24] [Anon-A], Authors’ prior work on multi-layer graph dependency analysis for publish–subscribe systems, citation withheld for double-anonymised review (n.d.).
- [25] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, IEEE Transactions on Software Engineering 22 (10) (1996) 751–761.
- [26] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: Proc. 27th Int. Conf. on Software Engineering (ICSE), 2005, pp. 580–586.
- [27] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE), 2007.
- [28] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, IEEE Transactions on Software Engineering 33 (1) (2007) 2–13.
- [29] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA), LNCS 5581, 2009, pp. 146–162.
- [30] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, IEEE Software 35 (3) (2018) 56–62.
- [31] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, Journal of Systems and Software 101 (2015) 193–220.
- [32] J. Humble, D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation, Addison-Wesley, 2010.
- [33] L. Chen, Continuous delivery: Huge benefits, but challenges too, IEEE Software 32 (2) (2015) 50–54.
- [34] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [35] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech.

- rep., International Organization for Standardization (2012).
- [36] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [37] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [38] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [39] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [40] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [41] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [42] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, Datasets and Benchmarks Track, 2024, arXiv:2402.02827.
- [43] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [44] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [45] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [46] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [47] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [48] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [49] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [50] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [51] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 19620–19631.
- [52] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [53] G. Beliakov, A. Pradera, T. Calvo, *Aggregation functions: A guide for practitioners*, *Studies in Fuzziness and Soft Computing* 221 (2007).
- [54] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, *IEEE Transactions on Systems, Man, and Cybernetics* 18 (1) (1988) 183–190.
- [55] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [56] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).

- [57] R. C. Martin, *Agile Software Development: Principles, Patterns, and Practices*, Prentice Hall, 2003.
- [58] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [59] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [60] Team SimPy, *Simpy: Event discrete simulation for Python*, <https://simpy.readthedocs.io> (2020).
- [61] International Organization for Standardization, *ISO/IEC 25022:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of quality in use*, Tech. rep., International Organization for Standardization (2016).
- [62] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, T. Azumi, Autoware on board: Enabling autonomous vehicles with embedded systems, in: *Proc. ACM/IEEE 9th Int. Conf. on Cyber-Physical Systems (ICCPS)*, 2018, pp. 287–296.
- [63] Google Cloud Platform, *Online boutique: A cloud-native microservices demo application*, Software artifact (2024).
- [64] X. Zhou, X. Peng, T. Xie, J. Sun, C. Ji, W. Li, D. Ding, Fault analysis and debugging of microservice systems: Industrial survey, benchmark system, and empirical study, *IEEE Transactions on Software Engineering* 47 (2) (2021) 243–260.
- [65] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: *Proc. AAAI Conference on Artificial Intelligence*, 2018, pp. 3538–3545.
- [66] D. Chen, Y. Lin, W. Li, P. Li, J. Zhou, X. Sun, Measuring and relieving the over-smoothing problem for graph neural networks from the topological view, in: *Proc. AAAI Conference on Artificial Intelligence*, 2020, pp. 3438–3445.
- [67] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [68] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [69] C. Spearman, The proof and measurement of association between two things, *American Journal of Psychology* 15 (1) (1904) 72–101.
- [70] M. D. Morris, Factorial sampling plans for preliminary computational experiments, *Technometrics* 33 (2) (1991) 161–174.
- [71] F. Campolongo, J. Cariboni, A. Saltelli, An effective screening design for sensitivity analysis of large models, *Environmental Modelling & Software* 22 (10) (2007) 1509–1518.
- [72] A. Saltelli, M. Ratto, T. Andres, F. Campolongo, J. Cariboni, D. Gatelli, M. Saisana, S. Tarantola, *Global Sensitivity Analysis: The Primer*, John Wiley & Sons, 2008.
- [73] I. M. Sobol’, Sensitivity estimates for nonlinear mathematical models, *Mathematical Modelling and Computational Experiments* 1 (4) (1993) 407–414.